

Artificial Intelligence & Git/GitHub

Hands-on assessment based on real software-development situations

PART A — ARTIFICIAL INTELLIGENCE IN SOFTWARE DEVELOPMENT

This assessment evaluates the ability to use AI as a software-development assistant rather than simply copying AI-generated code.

Section 1 — Conceptual Understanding

1. What is Artificial Intelligence? Explain it using examples from software development.
Artificial intelligence is the technology that ensures computers perform tasks that normally require human intelligence. Some of these tasks include learning from data from data, recognizing patterns, understanding language, solving problems, making predictions, and making decisions. Taking an example, AI tools like GitHub Copilot can suggest or generate code while a developer is programming. AI can also help developers find bugs, automate testing, and detect security problems.
2. Explain the basic flow of an AI system: input, processing and output.
AI takes the data as an input which can be in any form such as text, images or numbers and can be extracted from web scrapping, Github or other sources. And AI analyses the input using trained models or a trained model. At the end it produces a result, such as an answer, prediction or recommendation. To take an example, in a spam email detector, the input is email, the AI processes the email for suspicious patterns, and the output is the prediction such as spam or not spam.
3. Differentiate between Narrow AI and General AI.
Narrow AI is basically system designed to perform specific tasks such as recommendation system, voice assistants, spam detection and so on. General AI is a hypothetical system which is capable of performing the broad range of intellectual tasks at a human-like level. It is basically much more of building human like robots.
4. What is prompt engineering? Explain why a detailed prompt generally produces more useful development output.
Prompt engineering is basically the process of writing clear and specific instructions for an AI system to get the desired result. So here, a detailed prompt generates more of the desired and useful development output because it gives the AI clear information about the programming language, requirements, and expected results. Let's take an example when creating a login page, a better prompt is, create a react login page with email and password fields, input validations, and an error message for incorrect login. This helps the AI generate more accurate and useful code.
5. Identify common risks associated with AI-generated code.
AI-generated code can have multiple risks, including security vulnerabilities, bugs, outdated methods, and privacy issues. AI may generate code that looks correct but contains errors or does not follow best security practices. Developers should always review and test AI-generated code before using it. Let's take an example, AI might create a login system that works but does not securely protect user passwords.
6. Why must a developer review and test AI-generated code before using it?

Developers need to review and test AI-generated code before using it because AI does not produce reliable or correct code as always. The code may have hidden errors, unnecessary functions, or problems that could cause the application to fail. Testing helps confirm that the code works as expected and is safe to use in the final software.

7. Explain the developer workflow: Understand → Prompt → Generate → Review → Test → Secure → Deploy. The developer workflow follows series of important steps. First, understand usually means clearly identifying the problem, requirement, and expected result. Prompt is giving the AI clear and detailed instruction about what needs to be created. Generate is when the AI produces the code based on those instructions. Review means the developer checks the generated code for errors, quality, and correctness. Test make sure that the code works well in different situations. Secure means checking and fixing security risks, such as unsafe data handling or vulnerabilities. Finally, deploy means releasing the completed application to a server or platform where users can access it. Let's take an example to understand this workflow, if you want to build a login page, all you do is to understand what the login page needs. Then you prompt the AI with those requirements. The AI generates the code. You review the code for mistakes, test whether login page works, secure it to protect usernames and passwords, and finally deploy it so users can access it. So the output of one step becomes the starting point for the next step.

Section 2 — Real-Time Client Requirement Analysis

Scenario: A client wants a Student Task Management System. Students should be able to create, update, delete and complete tasks.

Act as the developer receiving this requirement. Use an AI assistant to help convert the requirement into a development plan.

- Identify the main users and their responsibilities.
- Convert the requirement into functional features.
- Identify the likely frontend screens/components.
- Identify the backend APIs required.
- Identify the database entities and important fields.
- Ask AI to identify edge cases that a developer may initially miss.
- Review the AI response and remove or modify anything that does not make sense.

Student Task Management System — Development Plan

1. Main Users

- The main user is the student. Students can create tasks, view their tasks, update task details, mark tasks as completed, and delete tasks.

2. Functional Features

The system should allow students to:

- Create a new task.
- View all tasks.
- Edit an existing task.
- Delete a task.
- Mark a task as completed or incomplete.
- Set a due date and priority.

3. Frontend Screens/Components

- The frontend could include a login page, task dashboard, create-task form, edit-task form, task list, and task details. Each task can have Edit, Delete, and Complete buttons.

4. Backend APIs

The main APIs would be:

- POST /tasks - Create a task
- GET /tasks - Get all tasks
- GET /tasks/:id - Get one task
- PUT /tasks/:id - Update a task
- DELETE /tasks/:id - Delete a task
- PATCH /tasks/:id - Mark a task completed/incomplete

5. Database Entity

A Task entity could contain:

- id
- title
- description
- dueDate
- priority
- status
- createdAt
- updatedAt

If student accounts are required, a Student entity could contain id, name, email, and password.

6. AI Prompt for Edge Cases

I would ask AI:

- Review this Student Task Management System where students can create, update, delete, and complete tasks. What edge cases or problems should a developer consider? AI could identify cases such as empty task titles, invalid due dates, duplicate tasks, deleting a task that does not exist, unauthorized access to another student's tasks, and database/server errors.

7. Review of AI Response

After reviewing the AI suggestions, I would keep important cases such as input validation, invalid task IDs, authorization, and error handling. I would remove unnecessary features that were not requested by the client, such as teacher/admin management, notifications, or complex task sharing. These could be added later if the client requests them.

Section 3 — Real-Time AI-Assisted Development

Scenario: Your team has asked you to implement the Create Task API for the Student Task Management System.

Use an AI assistant to support the implementation, but do not blindly accept its output.

- Create a clear development prompt containing the technology stack, requirements, expected API behaviour and validation rules.
- Generate the required route, controller and model/schema structure.
- Implement the feature in your project.
- Review the generated code line by line.
- Modify the code where necessary.
- Test the API using a suitable API testing tool.
- Verify both successful and invalid requests.

Development Prompt

I used the following prompt with an AI assistant to develop the Create Task API:

Create a Create Task API for a Student Task Management System using Node.js, Express.js, MongoDB, and

Mongoose. The task should contain title, description, status, and dueDate. The title must be required and cannot be empty. The description should be optional. The status should only allow pending, in-progress, or completed, with pending as the default value. The dueDate should be a valid required date. Create the required model/schema, controller, and route. The API should return HTTP 201 when a task is successfully created, HTTP 400 for invalid input, and HTTP 500 for unexpected server errors. Do not expose unnecessary internal error information. Use CommonJS syntax.

Implementation

I created a Node.js and Express.js project and installed Express, Mongoose, and dotenv. I created Task.js inside the models folder, taskController.js inside the controllers folder, taskRoutes.js inside the routes folder, server.js, and a .env file.

The Task.js file defines the structure of a task, including title, description, status, and dueDate. The taskController.js file contains the logic for validating the request and creating a new task. The taskRoutes.js file defines the POST route for creating a task. The server.js file configures Express, connects the application to MongoDB Atlas, and loads the task routes.

Code Review and Modification

I reviewed the AI-generated code before using it in the project. I checked the schema fields, required fields, status validation, route configuration, MongoDB connection, HTTP status codes, and error handling. I made sure that the title cannot be missing or empty and that invalid requests return HTTP 400. I also verified that successful task creation returns HTTP 201 and unexpected server errors return HTTP 500 without exposing sensitive internal information.

API Testing

I tested the API using Postman. The endpoint used was:

POST `http://localhost:3000/api/tasks`

For the successful test, I provided a title, description, status, and due date. The API successfully created the task and returned HTTP 201 Created with the message "Task created successfully."

I also tested an invalid request by removing the required title field. The API rejected the request and returned HTTP 400 Bad Request with the message "Title is required."

Test Results

The successful request confirmed that the API can create and store a task in MongoDB correctly. The invalid request confirmed that the validation is working because a task cannot be created without a title.

```
PS C:\Users\ksula\OneDrive\Desktop\student-task-api> npm init -y
"version": "1.0.0",
"main": "index.js",
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC",
"description": ""
}
```

• PS C:\Users\ksula\OneDrive\Desktop\student-task-api> npm install express mongoose dotenv

added 87 packages, and audited 88 packages in 10s

30 packages are looking for funding
run `npm fund` for details

found 0 vulnerabilities

• PS C:\Users\ksula\OneDrive\Desktop\student-task-api> npm install --save-dev nodemon

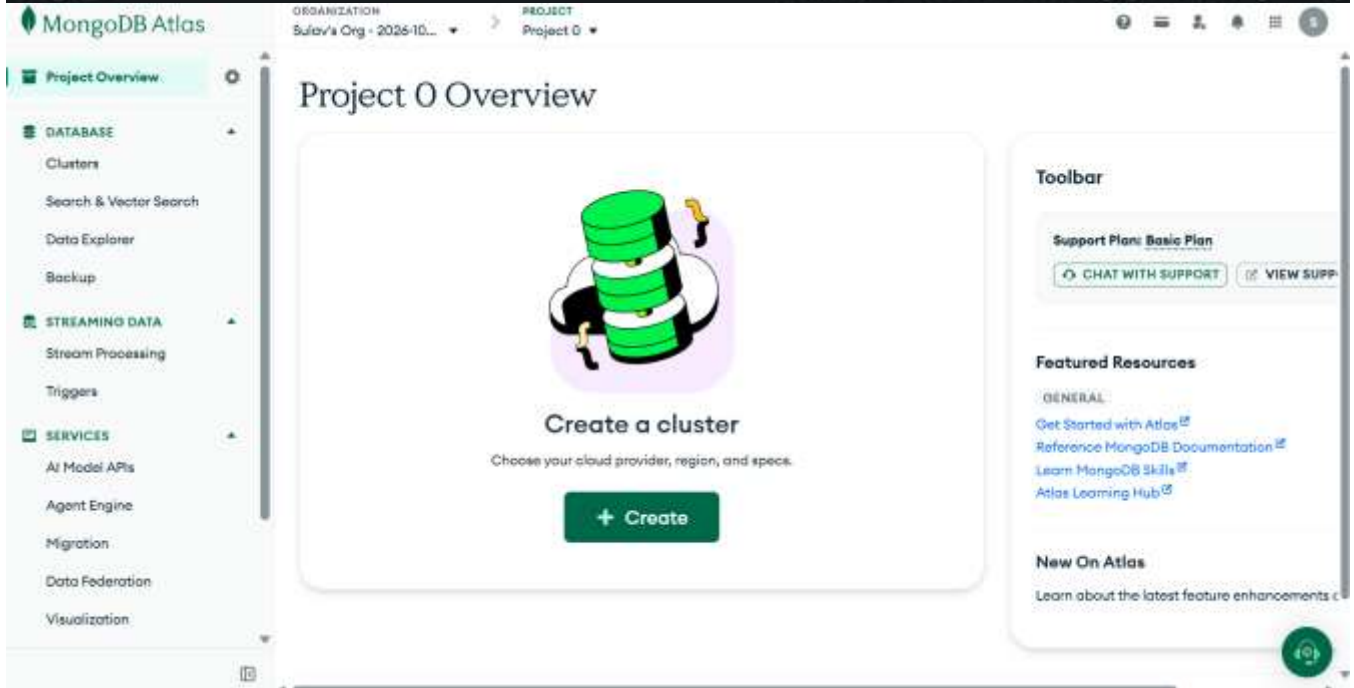
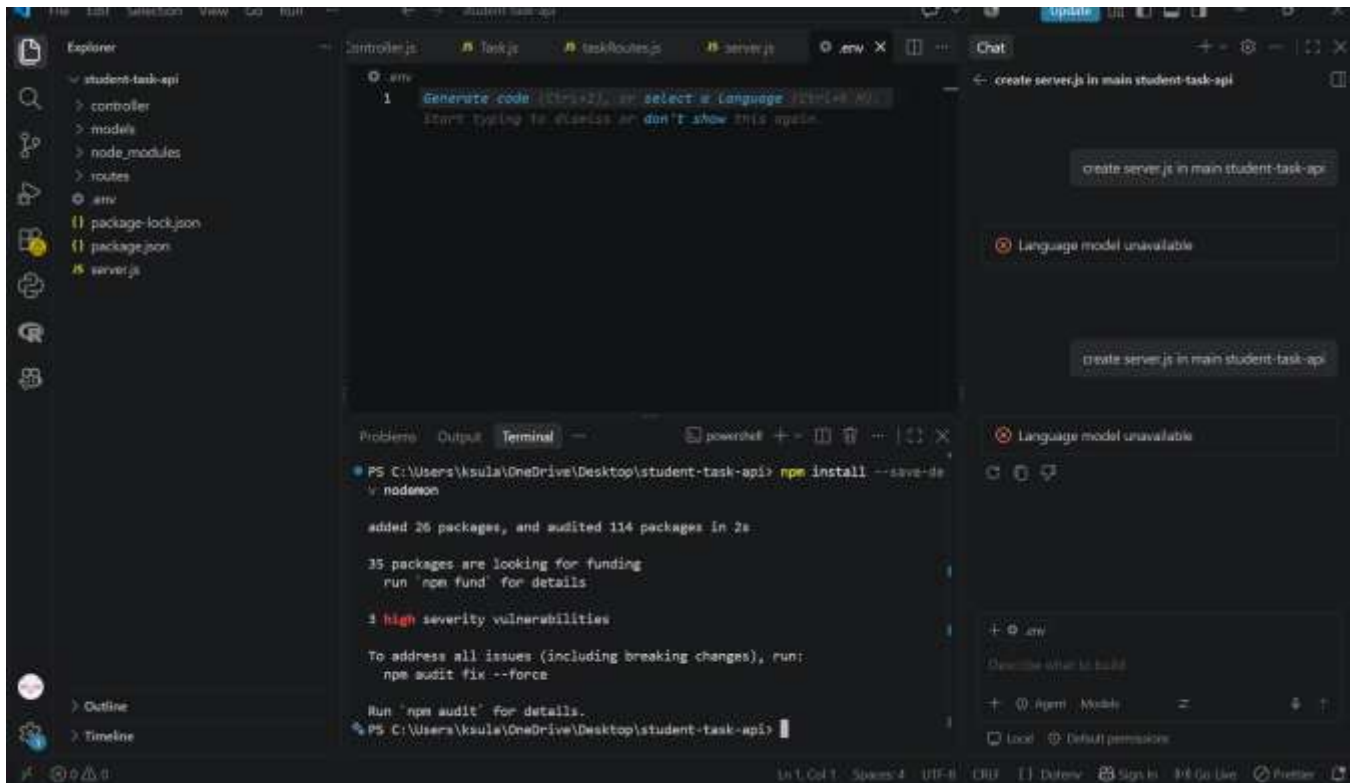
added 26 packages, and audited 114 packages in 2s

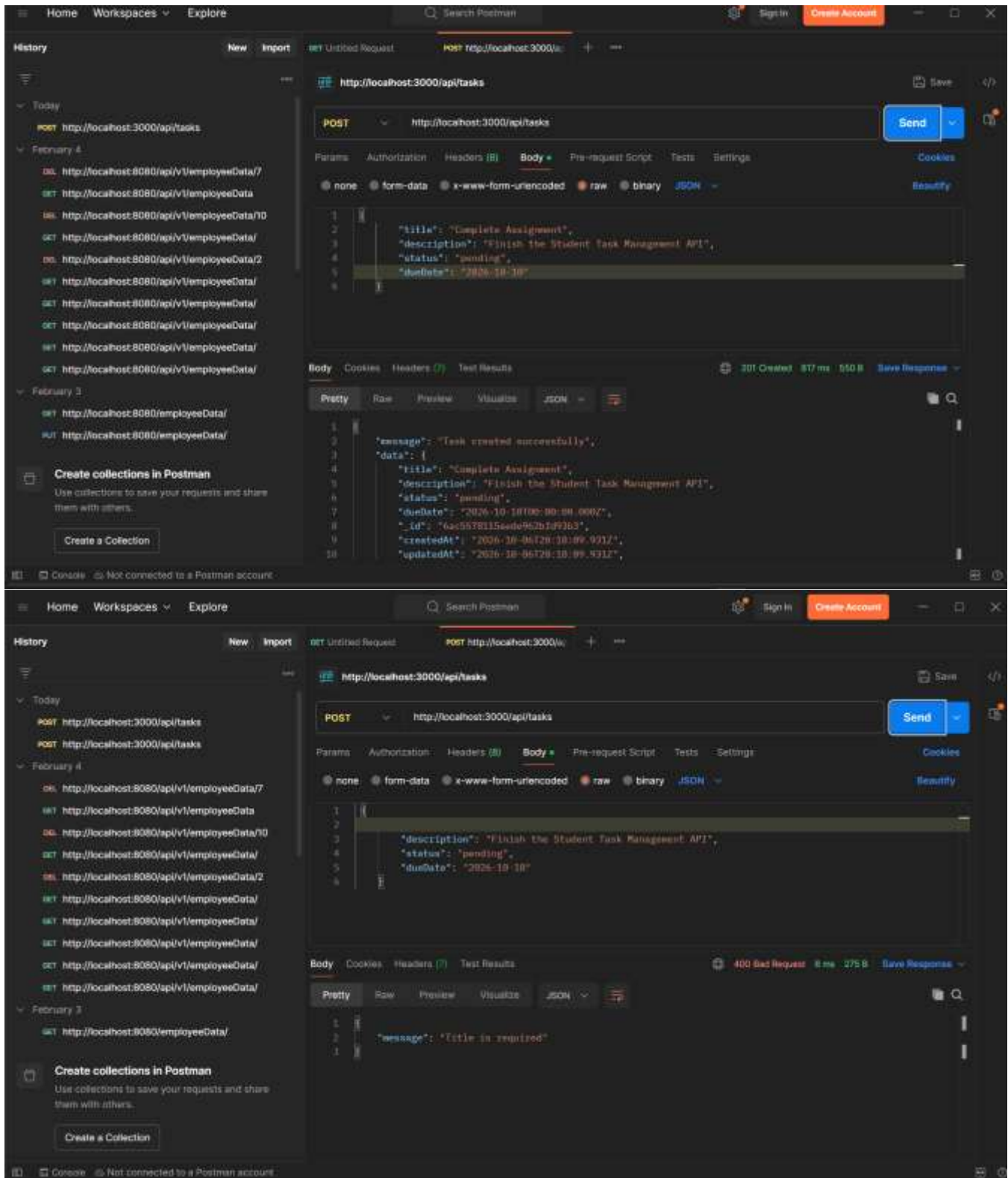
35 packages are looking for funding
run `npm fund` for details

3 **high** severity vulnerabilities

To address all issues (including breaking changes), run:
npm audit fix --force

Run `npm audit` for details.





Section 4 — Real-Time Debugging Challenge

Your team receives the following existing API code from another developer:

```
app.post("/tasks", async (req, res) => {

  try {
```

```

    const task = await Task.create(req.body);

    res.status(200).json({
      message: "Task created",
      data: tasks
    });
  } catch (error) {
    res.status(500).json(error);
  }
});

```

The API is not behaving correctly. Treat this as a real debugging ticket.

- Investigate the code yourself before asking AI for the solution.
After reviewing the code I found out that the created task is stored in the variable task, but the response uses tasks. Since the task is not defined, it causes an error.
- Create a debugging prompt that gives AI enough context to analyse the issue.
I have a Node.js/Express POST API that creates a task, but it returns an error. Review the code, identify the bug, explain the cause, and provide the corrected code.
- Compare the AI's explanation with your own investigation.
My investigation and the AI explanation found the same issue tasks should be task. AI also suggested using status code 201 because a new task is being created.
- Fix the problem.
The fixed code is below:

```

app.post("/tasks", async (req, res) => {
  try {
    const task = await Task.create(req.body);

    res.status(201).json({
      message: "Task created",
      data: task
    });
  } catch (error) {
    res.status(500).json(error);
  }
});

```

- Test the corrected API.
I tested the corrected API using Postman by sending a POST request with valid task data. The API returned 201 Created with the message “Task created” and the created task data. This confirmed that the corrected API was working successfully.
- Explain the root cause of the bug and why your correction solves it.
The bug was caused by using the wrong variable name. The code creates task but tries to return tasks. So, changing it to task fixes the error and returns the correct data.

Section 5 — Prompt Engineering in a Real Development Situation

Scenario: A team member writes the following prompt: “Create a login API.”

Improve this into a professional development prompt that an AI coding assistant can use effectively.

Your improved prompt should clearly communicate:

- Developer role/context
- Technology stack
- Functional requirements
- Input and output expectations
- Validation requirements
- Error handling
- Security requirements
- Constraints
- Expected response format

Then compare the result produced by the vague prompt with the result produced by your improved prompt.

“Create a login API.” This prompt is too vague because it does not provide enough information about the technology, requirements, validation, security, or expected output. So the improved prompt is:

Act as a backend developer and create a login API for a Student Task Management System using Node.js, Express.js, MongoDB, and JWT.

Create a POST /login endpoint that accepts an email and password. Both fields are required, and the email must be valid. Check the user in the database and securely compare the password using bcrypt.

For a successful login, return status 200 with a success message and JWT token. Return 400 for invalid or missing input, 401 for incorrect credentials, and 500 for server errors.

Passwords must never be returned or stored as plain text. JWT secrets should be stored in environment variables. Keep the code simple and do not add unnecessary features.

Provide the route, controller, and required model code separately with a short explanation.

In comparison, the vague prompt produces basic code, but AI has to guess the technology, authentication method, validation, and security requirements. The improved prompt gives clear instructions, so the result is more complete, secure, and suitable for the project.

Section 6 — AI Code Review & Security Challenge

Scenario: An AI assistant has generated authentication code for your application. Before the code is merged into the project, you are responsible for reviewing it.

- Ask AI to perform a code and security review.
- Identify issues involving validation, secrets, password handling, token handling, authorization or error handling.
- Do not assume every AI recommendation is correct. Verify each recommendation.
- Implement the necessary corrections.
- Test the authentication flow after making the corrections.
- Prepare a short explanation of the important changes you made.

1. AI Code Review Prompt

I asked AI to review the authentication code using this prompt: Review the authentication code for my Student Task Management System. The application uses Node.js, Express.js, MongoDB, Mongoose, bcryptjs, and JWT. Check the code for security problems related to input validation, passwords, secrets, JWT tokens, authorization, and error handling. Explain the problems and suggest how to fix them.

2. Issues Found

After reviewing the authentication code, I checked the AI suggestions before making changes. The important security areas were password storage, JWT secrets, token expiration, input validation, and error handling. Passwords should not be stored as normal text because anyone who gets access to the database could read them. JWT secrets should also not be written directly in the source code. Login errors should not tell someone whether a specific email exists. I also checked authorization and found that protected routes should verify the JWT token before allowing a user to access them.

3. Changes Made

I used bcryptjs to hash the password before saving it to MongoDB. I used bcryptjs to compare the entered password with the stored hashed password during login. I stored the JWT secret in the .env file instead of writing it directly in the JavaScript code. I also set the JWT token to expire after one hour. I added validation to make sure the required information is provided. The password must contain at least 8 characters. I also changed the login error handling so that both an incorrect email and an incorrect password return the same message: "Invalid email or password." This helps prevent someone from finding out which email accounts exist. The password is also not included in the registration response.

4. Authentication Testing

I tested the authentication API using Postman. First, I tested user registration. The user was successfully created and the API returned 201 Created. The response contained the user's ID, name, and email, but it did not return the password. Next, I tested login using the correct email and password. The API returned 200 OK with the message "Login successful" and generated a JWT token. Finally, I tested login with an incorrect password. The API returned 401 Unauthorized with the message "Invalid email or password."

5. Verification of AI Recommendations

I did not automatically accept every recommendation provided by AI. I reviewed the suggestions and checked them against the code before making changes. I verified that bcryptjs was used to hash passwords, the JWT secret was stored in the .env file, and the token had a one-hour expiration time. I also used Postman to verify that registration and login worked correctly after the security changes.

6. Important Changes

The most important changes were hashing passwords instead of storing them as plain text, keeping the JWT secret outside the source code, adding a one-hour expiration time to JWT tokens, validating required user information, and using a general error message for incorrect login information. These changes made the authentication system more secure.

```
PS C:\Users\ksula\OneDrive\Desktop\student-task-api> npm install bcryptjs
jsonwebtoken
```

```
added 14 packages, and audited 128 packages in 23s
```

```
36 packages are looking for funding
  run `npm fund` for details
```

```
3 high severity vulnerabilities
```

```
To address all issues (including breaking changes), run:
  npm audit fix --force
```

```
Run `npm audit` for details.
```

Explorer

- ▼ student-task-api
 - ▼ controller
 - JS authController.js
 - JS taskController.js
 - ▼ models
 - JS Task.js
 - JS User.js
 - > node_modules
 - ▼ routes
 - JS authRoutes.js
 - JS taskRoutes.js
 - .env
 - { } package-lock.json
 - { } package.json
 - JS server.js

JS taskController.js

.env

JS taskRoutes.js

JS server.js > ...

```
6   const authRoutes = require("./authRoutes");
7
8   const app = express();
9   const port = process.env.PORT || 3000;
10
11  app.use(express.json());
12  app.use("/api/tasks", taskRoutes);
13  app.use("/api/auth", authRoutes);
14
15  const startServer = async () => {
16    try {
17      await mongoose.connect(process.env.MONGO_URI);
18      app.listen(port, () => {
19        console.log(`Server is running on port ${port}`);
20      });
21    } catch (error) {
22      console.error(`Error connecting to MongoDB: ${error}`);
23    }
24  };
25
26  startServer();
```

Problems

Output

Debug Console

Terminal

```
PS C:\Users\ksula\OneDrive\Desktop\student-task-api>
```

History New Import GET Untitled Request POST http://localhost:3000/api/auth/register

Today

- POST http://localhost:3000/api/auth/register
- POST http://localhost:3000/api/tasks
- POST http://localhost:3000/api/tasks

February 4

- 04c http://localhost:8080/api/v1/employeeData/7
- 04c http://localhost:8080/api/v1/employeeData
- 04c http://localhost:8080/api/v1/employeeData/10
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/2
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/

February 3

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

POST http://localhost:3000/api/auth/register

Params Authorization Headers (0) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Test Student",
3   "email": "student@example.com",
4   "password": "SecurePass123!"
5 }
```

Body Cookies Headers (7) Test Results 201 Created 1129 ms 378 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "User registered successfully",
3   "user": {
4     "id": "6ac5458a248b3f5a6938f8fa",
5     "name": "Test Student",
6     "email": "student@example.com"
7   }
8 }
```

History New Import GET Untitled Request POST http://localhost:3000/api/auth/register POST http://localhost:3000/api/auth/login

Today

- POST http://localhost:3000/api/auth/login
- POST http://localhost:3000/api/auth/register
- POST http://localhost:3000/api/auth/login
- GET http://localhost:3000/api/auth/login
- POST http://localhost:3000/api/auth/register
- POST http://localhost:3000/api/tasks
- POST http://localhost:3000/api/tasks

February 4

- 04c http://localhost:8080/api/v1/employeeData/7
- 04c http://localhost:8080/api/v1/employeeData
- 04c http://localhost:8080/api/v1/employeeData/10
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/2
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/
- 04c http://localhost:8080/api/v1/employeeData/

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

POST http://localhost:3000/api/auth/register

Params Authorization Headers (0) Body Pre-request Script Tests Settings Cookies Beautify

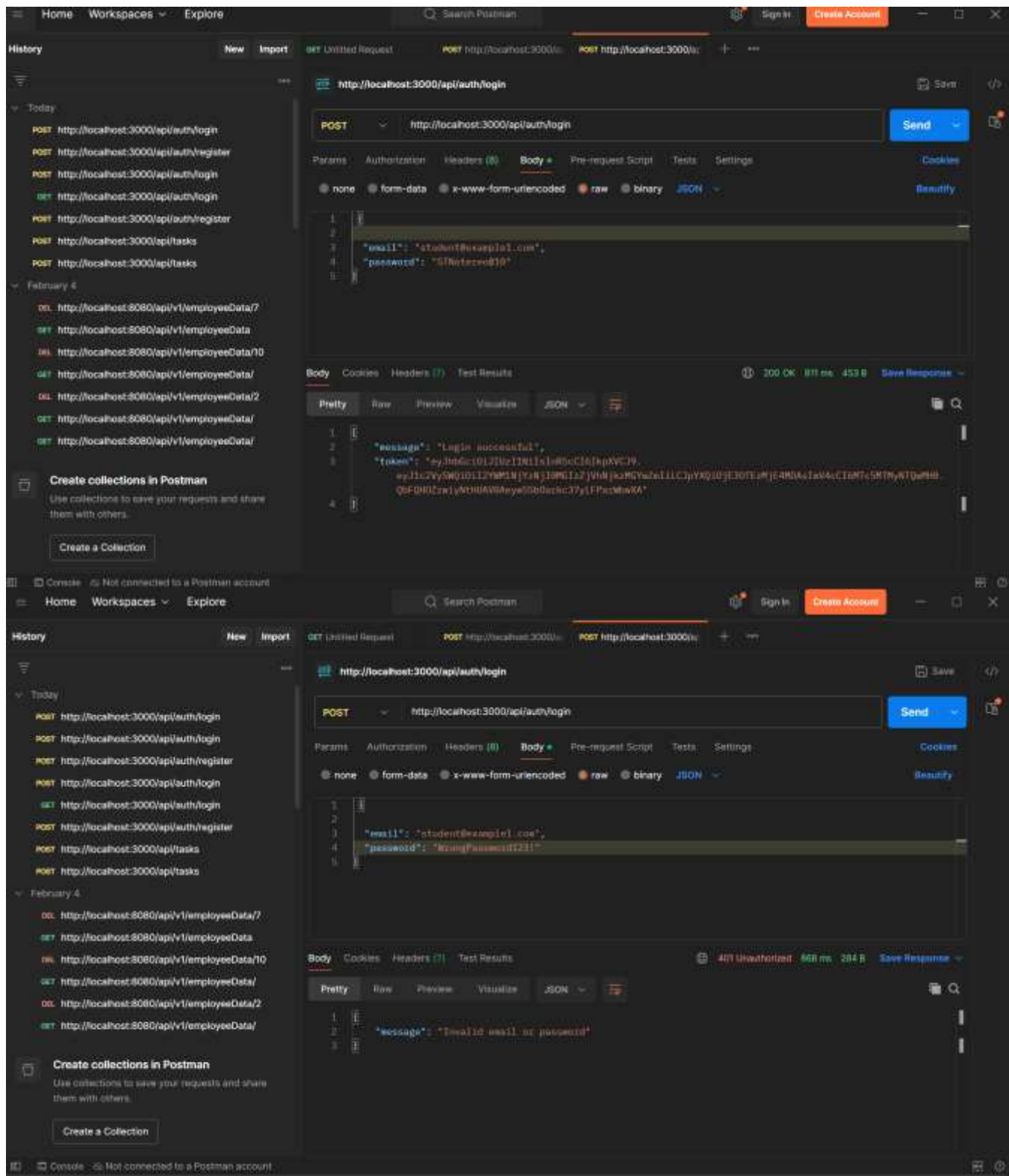
none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Test Student1",
3   "email": "student@example1.com",
4   "password": "SecurePass123!"
5 }
```

Body Cookies Headers (7) Test Results 201 Created 636 ms 378 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "User registered successfully",
3   "user": {
4     "id": "6ac56036248b3f5a6938f8fa",
5     "name": "Test Student1",
6     "email": "student@example1.com"
7   }
8 }
```



Section 7 — Final Real-Time AI Development Challenge

Scenario: The product owner asks your team to add a Search and Filter feature to the Student Task Management System.

Complete the feature using the following professional workflow:

Development Stage	What You Must Demonstrate
-------------------	---------------------------

Understand	Explain the requirement and identify assumptions.
Prompt	Write a clear, structured AI prompt.
Generate	Use AI to support the implementation.
Review	Inspect and improve the generated solution.
Test	Test normal, invalid and edge-case scenarios.
Secure	Check the feature for obvious security and validation issues.
Final Output	Demonstrate the working feature and explain your implementation.

Understand: The requirement was to add a Search and Filter feature to the Student Task Management System. The feature allows users to search for tasks by title and filter tasks by status. I assumed that the search should be case-insensitive and that the valid task statuses are pending, in-progress, and completed. I also assumed that search and filter should work separately or together. If no matching task is found, the API should return an empty array instead of an error.

Prompt: I used the following prompt with AI: I am developing a Student Task Management System using Node.js, Express.js, MongoDB, and Mongoose. I need to implement a Search and Filter feature for tasks. Create an Express API endpoint that allows users to search tasks by title using a query parameter called search and filter tasks by status using a query parameter called status. Valid statuses are pending, in-progress, and completed. The search should be case-insensitive, and search and status should be able to work together. Validate the status parameter and return 400 Bad Request for an invalid status. Handle errors properly and do not expose internal error details.

Generate: AI helped generate the searchTasks controller and the required route. The controller reads the search and status query parameters and creates a MongoDB filter. The search uses a case-insensitive regular expression to find matching task titles. The status parameter is used to return tasks with the selected status. The route was connected using: GET /api/tasks. The API can then be used with query parameters such as: /api/tasks?search=assignment, /api/tasks?status=pending and /api/tasks?search=assignment&status=pending

Review: I reviewed the AI-generated code before using it. I checked that the status values matched the statuses used by the Task model. I also checked that the search was case-insensitive and that search and filter could be used together. I improved the generated solution by trimming the search input and escaping special regular-expression characters before using the search value. I also made sure that an invalid status returns a clear 400 Bad Request response and unexpected server errors do not expose internal information.

Test: I tested the Search and Filter API using Postman with normal, invalid, and edge-case scenarios.

Search Test: I sent a GET request using search=assignment. The API returned 200 OK and returned the task titled "Complete Assignment."

Filter Test: I sent a GET request using status=pending. The API returned 200 OK and returned tasks with the pending status.

Combined Search and Filter Test: I used search=assignment&status=pending. The API returned 200 OK and returned the task that matched both the title search and status filter.

Invalid Status Test: I tested the API with status=wrong. Because wrong is not a valid task status, the API returned 400 Bad Request with the message "Invalid status".

Edge-Case Test:

I searched for xyz123notfound, which did not match any task. The API correctly returned 200 OK with an empty data array. This shows that a valid search with no matching results is handled correctly.

Secure: I reviewed the feature for security and validation issues. The status value is checked against the allowed statuses before it is used. The search input is trimmed, and special regular-expression characters are escaped before the value is passed to MongoDB. This reduces problems caused by unexpected search input. The API also returns a general "Internal server error" message for unexpected errors instead of exposing database or server details.

Final Output: The Search and Filter feature was successfully implemented in the Student Task Management System. Users can search tasks by title, filter tasks by status, or use both options together. I reviewed and improved the AI-generated solution instead of accepting it without checking. I then tested normal, invalid, and edge-case scenarios using Postman. All of the tests produced the expected results, showing that the feature works correctly and handles invalid input appropriately.

Home Workspaces Explore Search Postman Sign In Create Account

History New Import GET Unlabeled Request POST http://localhost:3000/... GET http://localhost:3000/api/...

Today

- GET http://localhost:3000/api/tasks?search=assignment
- POST http://localhost:3000/api/auth/login
- POST http://localhost:3000/api/auth/login
- POST http://localhost:3000/api/auth/register
- POST http://localhost:3000/api/auth/login
- GET http://localhost:3000/api/auth/login
- POST http://localhost:3000/api/auth/register
- POST http://localhost:3000/api/tasks
- POST http://localhost:3000/api/tasks

February 4

- OK http://localhost:8080/api/v1/employeeData/7
- GET http://localhost:8080/api/v1/employeeData
- OK http://localhost:8080/api/v1/employeeData/10
- GET http://localhost:8080/api/v1/employeeData/
- OK http://localhost:8080/api/v1/employeeData/2

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

GET http://localhost:3000/api/tasks?search=assignment

Params Authorization Headers (0) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value
search	assignment

Body Cookies Headers (7) Test Results 200 OK · 142 ms · 509 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "data": {
3     "id": "aec5578115aede962bf493b3",
4     "title": "Complete Assignment",
5     "description": "Finish the Student Task Management API",
6     "status": "pending",
7     "dueDate": "2026-10-10T00:00:00.000Z",
8     "createdAt": "2026-10-06T20:18:09.931Z",
9     "updatedAt": "2026-10-06T20:18:09.931Z"
10  }
```

Home Workspaces Explore Search Postman Sign In Create Account

History New Import GET Unlabeled Request POST http://localhost:3000/... GET http://localhost:3000/api/...

Today

- GET http://localhost:3000/api/tasks?status=pending
- GET http://localhost:3000/api/tasks?search=assignment
- POST http://localhost:3000/api/auth/login
- POST http://localhost:3000/api/auth/login
- POST http://localhost:3000/api/auth/register
- POST http://localhost:3000/api/auth/login
- GET http://localhost:3000/api/auth/login
- POST http://localhost:3000/api/auth/register
- POST http://localhost:3000/api/tasks
- POST http://localhost:3000/api/tasks

February 4

- OK http://localhost:8080/api/v1/employeeData/7
- GET http://localhost:8080/api/v1/employeeData
- OK http://localhost:8080/api/v1/employeeData/10
- GET http://localhost:8080/api/v1/employeeData/

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

GET http://localhost:3000/api/tasks?status=pending

Params Authorization Headers (0) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value
status	pending

Body Cookies Headers (7) Test Results 200 OK · 56 ms · 509 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "data": {
3     "id": "aec5578115aede962bf493b3",
4     "title": "Complete Assignment",
5     "description": "Finish the Student Task Management API",
6     "status": "pending",
7     "dueDate": "2026-10-10T00:00:00.000Z",
8     "createdAt": "2026-10-06T20:18:09.931Z",
9     "updatedAt": "2026-10-06T20:18:09.931Z"
10  }
```

HomeWorkspacesExplore

Search Postman

Sign InCreate Account

History

NewImport

GET http://localhost:3000/api/tasks?search=assignment&status=pending

GET http://localhost:3000/api/tasks?search=assignment&status=pending

GET http://localhost:3000/api/tasks?search=assignment&status=pending

POST http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/register

POST http://localhost:3000/api/auth/login

GET http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/register

POST http://localhost:3000/api/tasks

POST http://localhost:3000/api/tasks

February 4

GET http://localhost:8080/api/v1/employeeData/7

GET http://localhost:8080/api/v1/employeeData

GET http://localhost:8080/api/v1/employeeData/10

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

GET http://localhost:3000/api/tasks?search=assignment&status=pending

GET

http://localhost:3000/api/tasks?search=assignment&status=pending

Send

ParamsAuthorizationHeaders (0)BodyPre-request ScriptTestsSettingsCookies

Query Params

Key	Value
☒ search	assignment
☒ status	pending

BodyCookiesHeaders (0)Test Results

200 OK51 ms519 BSave Response

PrettyRawPreviewVisualizeJSON

```
1 {
2   "data": {
3     "id": "6ac5578115aed962b1d93b3",
4     "title": "Complete Assignment",
5     "description": "Finish the Student Task Management API",
6     "status": "pending",
7     "dueDate": "2026-10-10T00:00:00Z",
8     "createdAt": "2026-10-06T18:10:09.931Z",
9     "updatedAt": "2026-10-06T18:10:09.931Z",
10  }
11 }
```

HomeWorkspacesExplore

Search Postman

Sign InCreate Account

History

NewImport

GET http://localhost:3000/api/tasks?status=wrong

GET http://localhost:3000/api/tasks?status=wrong

GET http://localhost:3000/api/tasks?status=wrong

POST http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/register

POST http://localhost:3000/api/auth/login

GET http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/register

POST http://localhost:3000/api/tasks

POST http://localhost:3000/api/tasks

February 4

GET http://localhost:8080/api/v1/employeeData/7

GET http://localhost:8080/api/v1/employeeData

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

GET http://localhost:3000/api/tasks?status=wrong

GET

http://localhost:3000/api/tasks?status=wrong

Send

ParamsAuthorizationHeaders (0)BodyPre-request ScriptTestsSettingsCookies

Query Params

Key	Value
☒ status	wrong

BodyCookiesHeaders (0)Test Results

400 Bad Request21 ms272 BSave Response

PrettyRawPreviewVisualizeJSON

```
1 {
2   "message": "Invalid status"
3 }
```

HomeWorkspacesExplore

Search Postman

Sign InCreate Account

History

NewImport

GET http://localhost:3000/api/tasks?search=xyz123no

GET http://localhost:3000/api/tasks?status=wrong

GET http://localhost:3000/api/tasks?search=assignme

GET http://localhost:3000/api/tasks?status=pending

GET http://localhost:3000/api/tasks?search=assignme

POST http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/register

POST http://localhost:3000/api/auth/login

GET http://localhost:3000/api/auth/login

POST http://localhost:3000/api/auth/register

POST http://localhost:3000/api/tasks

POST http://localhost:3000/api/tasks

February 4

GET http://localhost:8080/api/v1/employeeData/7

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

GET http://localhost:3000/api/tasks?search=xyz123notfound

Send

ParamsAuthorizationHeaders (6)BodyPre-request ScriptTestsSettingsCookies

Query Params

Key	Value
search	xyz123notfound

BodyCookiesHeaders (7)Test Results

200 OK48 ms345 BSave Response

PrettyRawPreviewVisualizeJSON

```
1 {
2   "data": []
3 }
```

Console

Not connected to a Postman account

PART B — GIT & GITHUB

This assessment evaluates Git through real repository operations. Students should work through the scenarios as if they were collaborating on a live software project.

Section 1 — Conceptual Understanding

8. What is Git and why is it used in software development?
Git is version control system used to track changes in code during software development. It allows developers to save different versions of their work or code, work together on the same project, create branches for new features and return to previous versions if something goes wrong.
9. What is the difference between Git and GitHub?
Git is the version control system used to track and manage changes to the code on your computer. So , GitHub is basically an online platform to stores Git repositories and allows developers to share code, collaborate with others, and manage projects.
10. Explain HEAD, HEAD~1 and HEAD~2.
HEAD represents the current commit you are working on in Git. HEAD~1 refers to one commit before the current commit, while HEAD~2 refers to two commits before the current commit. These are useful when viewing or going back to previous versions of your code.
11. Explain the difference between git restore, git reset and git revert.
The git restore is used to undo changes in files. git reset moves the branch back to a previous commit and can remove commits from history. git revert safely undoes a previous commit by creating a new commit that reverses its changes.
12. Explain the difference between git fetch and git pull.
The git fetch downloads the latest changes from the remote repository but does not change your local code. git pull downloads the latest changes and automatically merges them into your current branch.
13. What is a branch and why is branching useful in team development?
A branch is a separate version of the code where developers can work without affecting the main project. Branching is useful in team development because multiple developers can work on different features or fixes at the same time and merge their changes later.
14. What is a merge conflict and how is it resolved?
A merge conflict happens when Git cannot automatically combine changes from different branches, usually because the same part of a file was changed differently. It is resolved by reviewing the conflicting code, choosing the correct changes, saving the file, and then committing the resolved version.
15. What is git reflog and when would you use it?
The git reflog shows a history of where HEAD and branches have pointed in your local Git repository. It is useful for recovering lost commits or changes after actions like git reset, rebase, or accidentally deleting a branch.

Section 2 — Real-Time Project Repository Setup

Scenario: You have joined a development team and have been asked to create the local Git repository for a new web project.

Create a small project folder and perform the complete initial Git workflow.

```
git init
git status
git add .
git commit -m "Initial project setup"
```

```
git log
```

Then make another meaningful change and create another commit.

- Use git status before and after staging.
- Use git log to inspect the commit history.
- Use git diff to inspect changes before committing.
- Use git diff --staged to inspect staged changes.

1. Initialize the Git Repository

I created a folder named student-web-project and created an index.html file inside it. Then I opened the terminal in the project folder and used the git init command. This command created a new local Git repository for my project.

2. Check Git Status

I used the git status command to check the current status of my project. It showed that index.html was an untracked file. This means Git could see the file, but the file was not added to Git yet.

3. Stage the File

I used the git add . command to stage the index.html file. After that, I used git status again. It showed index.html under "Changes to be committed," which confirmed that the file was successfully staged.

4. Create the First Commit

I used git commit -m "Initial project setup" to create my first commit. After that, I used git log to check the commit history. The result showed that my first commit was successfully created.

5. Make a Change to the Project

I made another change to my index.html file by adding an About section. Before staging the change, I used git status to check the modified file. I also used git diff to see exactly what was changed in the file.

6. Stage and Review the Changes

I used git add . to stage the new changes. Then I used git status to confirm that the changes were staged. I also used git diff --staged to review the changes before creating the second commit.

7. Create the Second Commit

I created my second commit using git commit -m "Add about section". After that, I used git status to check the repository. It showed "nothing to commit, working tree clean," which means all changes were successfully committed.

Finally, I used git log --oneline to check the commit history. It showed both "Add about section" and "Initial project setup" commits.

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git init
Initialized empty Git repository in C:/Users/ksula/OneDrive/Desktop/student-web-project/.git/
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> █
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add .
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

● PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Initial project setup"
[master (root-commit) f6106e3] Initial project setup
 1 file changed, 12 insertions(+)
 create mode 100644 index.html
● PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log
commit f6106e34b3e3c740fee7b069c34c711c3fc8ab3c (HEAD -> master)
Author: Sulav <ksulav10@gmail.com>
Date:   Tue Oct 6 10:06:59 2026 -0500

    Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch master

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git diff
diff --git a/index.html b/index.html
index c78e547..a54b397 100644
--- a/index.html
+++ b/index.html
@@ -6,7 +6,7 @@
     <title>Student Web Project</title>
   </head>
   <body>
-     <h1>Welcome to My Web Project</h1>
-     <p>This project is created to practice Git.</p>

```

```

+++ b/index.html
@@ -6,7 +6,7 @@
    <title>Student Web Project</title>
  </head>
  <body>
-   <h1>Welcome to My Web Project</h1>
+   <p>This project is created to practice Git.</p>
+   <h2>About This Project</h2>
+   <p>This project demonstrates the basic Git workflow.</p>
  </body>
</html>
(END)

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Add about section"
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline
f6106e3 (HEAD -> master) Initial project setup
• PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline
f6106e3 (HEAD -> master) Initial project setup
• PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add .
• PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html
• PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html
• PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git diff --staged
diff --git a/index.html b/index.html
index c78e547..a54b397 100644
--- a/index.html
+++ b/index.html
@@ -6,7 +6,7 @@
    <title>Student Web Project</title>
  </head>
  <body>
-   <h1>Welcome to My Web Project</h1>
-   <p>This project is created to practice Git.</p>
+   <h2>About This Project</h2>
+   <p>This project demonstrates the basic Git workflow.</p>
  </body>
</html>
• PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Add about section"
[master d6a14c8] Add about section
 1 file changed, 2 insertions(+), 2 deletions(-)
• PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch master
nothing to commit, working tree clean
• PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline
d6a14c8 (HEAD -> master) Add about section

```

Section 3 — Real-Time Feature Branch Workflow

Scenario: Your team follows a branch-based workflow. You are responsible for implementing the login feature without directly working on the main branch.

```
git switch -c development
```

```
git switch -c feature-login
```

- Create a login-related change.
- Commit the change on feature-login.
- Switch between branches and verify which files/commits are present.
- Use git branch to inspect available local branches.
- Demonstrate git switch - to move back to the previous branch.

1. Create the Development Branch

I created a new development branch using git switch -c development. I used git branch to check the available branches and confirmed that I was on the development branch.

2. Create the Feature Login Branch

I created a new branch called feature-login using git switch -c feature-login. I used git branch again to check my branches. The star (*) next to feature-login showed that I was working on the feature-login branch.

3. Create the Login Feature

I created a new file called login.html on the feature-login branch. I added a simple login form with email, password, and login button fields. I used git status to check the new file.

4. Commit the Login Feature

I staged the login.html file and created a commit with the message “Add login feature.” I used git log --oneline to check the commit history and confirmed that the login feature was committed on the feature-login branch.

5. Switch to the Development Branch

I switched from feature-login to development. I used git branch to confirm that I was on the development branch. I also used git log --oneline and noticed that the “Add login feature” commit was not present on the development branch because the login feature was created only on feature-login.

6. Use git switch -

I used git switch - to return to my previous branch. It switched me from development back to feature-login. I used git branch to confirm the current branch and git log --oneline to check the commit history. The “Add login feature” commit appeared again.

```
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c development
Switched to a new branch 'development'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
* development
  master
  main
  ~
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c feature-login
Switched to a new branch 'feature-login'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
  development
* feature-login
  master
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch feature-login
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    login.html

nothing added to commit but untracked files present (use "git add" to track)
```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add .
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch feature-login
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   login.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Add login feature"
[feature-login a40d8b6] Add login feature
 1 file changed, 17 insertions(+)
 create mode 100644 login.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline
a40d8b6 (HEAD -> feature-login) Add login feature
d6a14c8 (master, development) Add about section
f6106e3 Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
* development
  feature-login
  master
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline
d6a14c8 (HEAD -> development, master) Add about section
f6106e3 Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -
Switched to branch 'feature-login'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
  development
* feature-login
  master
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline
a40d8b6 (HEAD -> feature-login) Add login feature
d6a14c8 (master, development) Add about section
f6106e3 Initial project setup

```

Section 4 — Real-Time Merge Conflict

Scenario: Two developers have edited the same part of a project. You are responsible for integrating the feature branch into the main development line.

- Create two branches that modify the same file or line in different ways.
- Switch to the branch that should receive the changes.
- Merge the feature branch.
- Identify the merge conflict.
- Open the conflicted file and understand the conflict markers.
- Resolve the conflict without accidentally deleting valid work.
- Stage the resolved file.
- Complete the merge commit.
- Inspect the final history.
- Demonstrate git merge --abort in a separate conflict situation and explain when it is useful.

For this task, I created two separate branches called developer-one and developer-two. Both branches changed the same part of the index.html file in different ways. I committed the changes separately on each branch. Then, while working on the developer-two branch, I tried to merge the developer-one branch. Git could not automatically combine the changes because the same line had been modified differently on both branches, so it displayed a merge conflict.

I opened the conflicted index.html file and reviewed the conflict markers. The <<<<<< marker showed the change from the current branch, the ===== marker separated the two changes, and the >>>>>> marker showed the incoming change. I reviewed the changes and manually resolved the conflict while keeping the valid project content. After resolving it, I staged the file and completed the merge commit. I then used git log --oneline --graph --all to inspect the final history and confirm that the branches were successfully merged.

I also created another conflict using two separate branches called abort-one and abort-two. Both branches changed the same line in index.html differently. When I tried to merge abort-one into abort-two, Git reported a merge conflict. Instead of resolving this conflict, I used `git merge --abort`. This cancelled the merge and returned the abort-two branch to the state it was in before the merge attempt. I used `git status` afterward, and it showed “nothing to commit, working tree clean.” This demonstrated that `git merge --abort` is useful when I want to cancel an unfinished merge and return to the previous working state.

```
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch development
nothing to commit, working tree clean
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c developer-one
Switched to a new branch 'developer-one'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add .
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Update heading by developer two"
[developer-one 63af47f] Update heading by developer two
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Update heading by developer one"
On branch developer-one
nothing to commit, working tree clean
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c developer-two
Switched to a new branch 'developer-two'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add .
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Update heading by developer two"
[developer-two 6e21ff3] Update heading by developer two
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
  developer-one
* developer-two
  development
  feature-login
  master
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c developer-two
Switched to a new branch 'developer-two'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add .
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Update heading by developer two"
[developer-two 6e21ff3] Update heading by developer two
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
  developer-one
* developer-two
  development
  feature-login
  master
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git merge developer-one
Merge made by the 'ort' strategy.
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline --graph --all
* 9d9f13b (HEAD -> developer-two) Merge branch 'developer-one' into developer-two
|
| * 63af47f (developer-one) Update heading by developer two
| * 6e21ff3 Update heading by developer two
|/
| * a40d8b6 (feature-login) Add login feature
|/
* d6a14c8 (master, development) Add about section
* f6106e3 Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c abort-one
Switched to a new branch 'abort-one'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add .
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Change project description for abort one"
[abort-one 795db3b] Change project description for abort one
1 file changed, 1 insertion(+), 1 deletion(-)
```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c abort-two
Switched to a new branch 'abort-two'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add .
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Change project description for abort two"
[abort-two fc439e7] Change project description for abort two
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git merge abort-one
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   index.html

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git merge --abort
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
nothing to commit, working tree clean

```

Section 5 — Real-Time Rollback & Recovery

Scenario: A developer makes several changes and then realizes that different levels of rollback are required. Handle each situation using the appropriate Git operation.

Real-Time Situation	Task
Uncommitted file changes	Modify a file and then discard the local modification using git restore.
Staged changes	Stage a file and then remove it from the staging area using git restore --staged.
Local commit	Create a local commit that should be removed or moved back, then demonstrate an appropriate git reset mode.
Shared commit	Create a commit that represents a change which should be undone without rewriting shared history. Demonstrate git revert.
Lost-looking commit	Use git reflog to locate a commit after history has been moved and demonstrate how the commit can be recovered.

During each scenario, explain why the selected command is appropriate.

In this task, I practiced different Git rollback and recovery methods. First, I modified index.html without committing it and used git restore to discard the local change. Next, I staged a change and used git restore --staged to remove it from the staging area while keeping the change in the file. I then created a temporary local commit and used git reset --soft HEAD~1 to remove the commit from the current history while keeping its changes staged. For a shared commit, I used git revert, which created a new commit that reversed the previous change without deleting the original commit from the history. Finally, I used git reflog to find the temporary commit that was no longer visible in the normal git log after the reset. I recovered it by creating a new branch called recovered-commit using its commit ID. This task helped

me understand when to use git restore, git restore --staged, git reset, git revert, and git reflow for different rollback and recovery situations.

```
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git restore index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
nothing to commit, working tree clean

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git restore --staged index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git restore index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
nothing to commit, working tree clean
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
nothing to commit, working tree clean
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Temporary local commit"
[abort-two 2978fdc] Temporary local commit
 1 file changed, 1 insertion(+)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -3
2978fdc (HEAD -> abort-two) Temporary local commit
fc439e7 Change project description for abort two
d6a14c8 (master, development) Add about section

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git reset --soft HEAD~1
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -3
fc439e7 (HEAD -> abort-two) Change project description for abort two
d6a14c8 (master, development) Add about section
f6106e3 Initial project setup

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html
```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git restore --staged index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git restore index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch abort-two
nothing to commit, working tree clean
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Add temporary shared change"
[abort-two 1878167] Add temporary shared change
1 file changed, 1 insertion(+)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -3
1878167 (HEAD -> abort-two) Add temporary shared change
fc439e7 Change project description for abort two
d6a14c8 (master, development) Add about section
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -3
1878167 (HEAD -> abort-two) Add temporary shared change
fc439e7 Change project description for abort two
d6a14c8 (master, development) Add about section
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git revert 1878167
[abort-two f260f82] Revert "Add temporary shared change"
1 file changed, 1 deletion(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -4
f260f82 (HEAD -> abort-two) Revert "Add temporary shared change"
1878167 Add temporary shared change
fc439e7 Change project description for abort two
d6a14c8 (master, development) Add about section

```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -4
f260f82 (HEAD -> abort-two) Revert "Add temporary shared change"
1878167 Add temporary shared change
fc439e7 Change project description for abort two
d6a14c8 (master, development) Add about section
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git relog
f260f82 (HEAD -> abort-two) HEAD@{0}: revert: Revert "Add temporary shared change"
1878167 HEAD@{1}: commit: Add temporary shared change
fc439e7 HEAD@{2}: reset: moving to HEAD~1
2978fdc HEAD@{3}: commit: Temporary local commit
fc439e7 HEAD@{4}: reset: moving to HEAD
fc439e7 HEAD@{5}: commit: Change project description for abort two
d6a14c8 (master, development) HEAD@{6}: checkout: moving from development to abort-two
d6a14c8 (master, development) HEAD@{7}: checkout: moving from abort-one to development
795db3b (abort-one) HEAD@{8}: commit: Change project description for abort one
d6a14c8 (master, development) HEAD@{9}: checkout: moving from development to abort-one
d6a14c8 (master, development) HEAD@{10}: checkout: moving from developer-two to development
0d0f13b (developer-two) HEAD@{11}: merge developer-one: Merge made by the 'ort' strategy.
6e21ff3 HEAD@{12}: commit: Update heading by developer two
d6a14c8 (master, development) HEAD@{13}: checkout: moving from development to developer-two
d6a14c8 (master, development) HEAD@{14}: checkout: moving from developer-one to development
63af47f (developer-one) HEAD@{15}: commit: Update heading by developer two
d6a14c8 (master, development) HEAD@{16}: checkout: moving from development to developer-one
d6a14c8 (master, development) HEAD@{17}: checkout: moving from feature-login to development
a40d8b6 (feature-login) HEAD@{18}: checkout: moving from development to feature-login
d6a14c8 (master, development) HEAD@{19}: checkout: moving from feature-login to development
a40d8b6 (feature-login) HEAD@{20}: commit: Add login feature
d6a14c8 (master, development) HEAD@{21}: checkout: moving from development to feature-login
:...skipping...
f260f82 (HEAD -> abort-two) HEAD@{0}: revert: Revert "Add temporary shared change"
1878167 HEAD@{1}: commit: Add temporary shared change
fc439e7 HEAD@{2}: reset: moving to HEAD~1
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch recovered-commit 2978fdc
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
  abort-one
* abort-two
  developer-one
  developer-two
  development
  feature-login
  master
  recovered-commit
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log recovered-commit --oneline -3
2978fdc (recovered-commit) Temporary local commit
fc439e7 Change project description for abort two
d6a14c8 (master, development) Add about section

```

Section 6 — Real-Time GitHub Collaboration

Scenario: Your local project now needs to be connected to the team's GitHub repository.

- Connect the local repository to the remote repository.
- Inspect the configured remote using `git remote -v`.
- Push the local branch to GitHub.
- Fetch updates from the remote repository.
- Display remote branches using `git branch -r`.
- Display both local and remote branches using `git branch -a`.
- Create a local tracking branch from a remote development branch.
- Pull the latest changes and verify the local repository state.

```
git remote -v
```

```
git fetch

git branch -r

git branch -a

git switch --track origin/development

git pull
```

In this task, I connected my local Git repository to a GitHub repository using a remote named origin. I used `git remote -v` to verify the remote connection and pushed the development branch to GitHub. I used `git fetch` to get information from the remote repository, `git branch -r` to view remote branches, and `git branch -a` to view both local and remote branches. I then created a local development branch that tracks origin/development using `git switch --track origin/development`. Finally, I used `git pull` to get the latest changes and `git status` to verify that my local development branch was up to date with the remote repository and the working tree was clean.

```
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git remote -v
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git remote add origin https://github.com/Sulav-ui/student-web-project.git
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git remote -v
origin https://github.com/Sulav-ui/student-web-project.git (fetch)
origin https://github.com/Sulav-ui/student-web-project.git (push)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git push -u origin development
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 738 bytes | 246.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Sulav-ui/student-web-project.git
 * [new branch]      development -> development
branch 'development' set up to track 'origin/development'.
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git fetch
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch -r
origin/development
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch -a
abort-one
abort-two
developer-one
developer-two
* development
feature-login
master
recovered-commit
remotes/origin/development
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch master
Switched to branch 'master'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch -m development development-old
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
abort-one
abort-two
developer-one
developer-two
development-old
feature-login
* master
recovered-commit
```

```

recovered commit
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch --track origin/development
branch 'development' set up to track 'origin/development'.
Switched to a new branch 'development'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git pull
Already up to date.
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch development
Your branch is up to date with 'origin/development'.

nothing to commit, working tree clean

```

Section 7 — Real-Time Stash Scenario

Scenario: You are halfway through a feature. Your manager reports an urgent production bug that must be fixed immediately. Your current feature is not ready to commit.

- Temporarily save your unfinished work using git stash.
- Switch to the appropriate branch.
- Make and commit the urgent bug fix.
- Return to your feature branch.
- Inspect the available stashes.
- Restore your unfinished work using git stash pop or another appropriate stash operation.
- Continue your feature without losing your previous changes.

In this task, I was working on an unfinished feature on the feature-profile branch when an urgent bug needed to be fixed. I used git stash to temporarily save my unfinished changes without committing them. I then worked on the urgent bug fix and committed it to the development branch. After completing the bug fix, I returned to the feature-profile branch and used git stash list to check my saved work. Finally, I used git stash pop to restore the unfinished changes. I checked git status and confirmed that my previous changes were restored successfully, allowing me to continue working on the feature without losing my work.

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c feature-profile
Switched to a new branch 'feature-profile'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch feature-profile
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash push -m "Unfinished profile feature"
Saved working directory and index state On feature-profile: Unfinished profile feature
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash list
stash@{0}: On feature-profile: Unfinished profile feature
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Fix urgent production bug"
[feature-profile 93e95d0] Fix urgent production bug
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch feature-profile
nothing to commit, working tree clean

```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch feature-profile
nothing to commit, working tree clean
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
Your branch is up to date with 'origin/development'.
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -3
d6a14c8 (HEAD -> development, origin/development, master, development-old) Add about section
f6106e3 Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git cherry-pick 93e95d0
[development b7784fd] Fix urgent production bug
Date: Tue Oct 6 13:35:45 2026 -0500
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -3
b7784fd (HEAD -> development) Fix urgent production bug
d6a14c8 (origin/development, master, development-old) Add about section
f6106e3 Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch feature-profile
Switched to branch 'feature-profile'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash list
stash@{0}: On feature-profile: Unfinished profile feature
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash pop
Auto-merging index.html
On branch feature-profile
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (02da5cfad3d9492386e0ce702132999d414e1d10)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch feature-profile
Switched to branch 'feature-profile'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash list
stash@{0}: On feature-profile: Unfinished profile feature
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash pop
Auto-merging index.html
On branch feature-profile
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (02da5cfad3d9492386e0ce702132999d414e1d10)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch feature-profile
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

```

Section 8 — Final Real-Time Git/GitHub Challenge

Scenario: You are working as a developer in a team. The project has a main branch and a development branch. You are assigned a new feature while another developer is working on a different feature.

- Start from the existing repository.
- Create or switch to the appropriate development branch.
- Create your own feature branch.
- Make multiple meaningful commits.
- Inspect changes using `git status`, `git diff` and `git log`.
- Push the feature branch to GitHub.
- Simulate another developer making changes.
- Fetch remote information and inspect remote branches.
- Merge the appropriate feature into the development workflow.
- Resolve a merge conflict if one occurs.
- Temporarily stash unfinished work when required.
- Demonstrate an appropriate rollback operation.
- Use `git reflog` to recover a commit in a controlled recovery scenario.
- Leave the repository in a clean and understandable final state.

In this final challenge, I used an existing Git repository and worked from the development branch. I created a feature-contact branch and made multiple commits for a contact page and contact form. I used `git status`, `git diff`, and `git log` to inspect my changes and then pushed the feature branch to GitHub. I also simulated another developer by creating a feature-footer branch and pushing it to the remote repository. I used `git fetch` and inspected the remote branches before merging the feature branches into development. I also practiced using `git stash` to temporarily save unfinished work. For rollback and recovery, I created a temporary commit, removed it using `git reset --hard`, located the commit using `git reflog`, and recovered it safely by creating a recovery branch. Finally, I verified the repository with `git status` to make sure the working tree was clean.

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash push -m "Save profile work before final challenge"
Saved working directory and index state On feature-profile: Save profile work before final challenge
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
Your branch is ahead of 'origin/development' by 1 commit.
(use "git push" to publish your local commits)

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 297 bytes | 148.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Sulav-ui/student-web-project.git
    d6a14c8..b7784fd  development -> development
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c feature-contact
Switched to a new branch 'feature-contact'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
  abort-one
  abort-two
  developer-one
  developer-two
  development
  development-old
* feature-contact
  feature-login
  feature-profile
  master
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch
  recovered-commit
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch feature-contact
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    contact.html

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git diff
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add contact.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Add contact page"
[feature-contact 2baf187] Add contact page
 1 file changed, 10 insertions(+)
 create mode 100644 contact.html

```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch feature-contact
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   contact.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git diff
diff --git a/contact.html b/contact.html
index 9f135e6..1a317b9 100644
--- a/contact.html
+++ b/contact.html
@@ -4,7 +4,10 @@
     <title>Contact Us</title>
   </head>
   <body>
-    <h2>Contact Us</h2>
-    <p>Email us at contact@example.com</p>
+    <form>
+      <input type="text" placeholder="Your Name" />
+      <input type="email" placeholder="Your Email" />
+      <button type="submit">Send</button>
+    </form>
   </body>

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add contact.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Add contact form"
[feature-contact b5f43f2] Add contact form
 1 file changed, 5 insertions(+), 2 deletions(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -5
b5f43f2 (HEAD -> feature-contact) Add contact form
2baf187 Add contact page
b7784fd (origin/development, development) Fix urgent production bug
d6a14c8 (master, development-old) Add about section
f6106e3 Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git push -u origin feature-contact
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 780 bytes | 260.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-contact' on GitHub by visiting:
remote:   https://github.com/Sulav-ui/student-web-project/pull/new/feature-contact
remote:
To https://github.com/Sulav-ui/student-web-project.git
 * [new branch]      feature-contact -> feature-contact
branch 'feature-contact' set up to track 'origin/feature-contact'.
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch -r
  origin/development
  origin/feature-contact

```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Your branch is up to date with 'origin/development'.
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch -c feature-footer
Switched to a new branch 'feature-footer'
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add index.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Add website footer"
[feature-footer b22ada5] Add website footer
1 file changed, 1 insertion(+), 2 deletions(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git push -u origin feature-footer
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 336 bytes | 336.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature-footer' on GitHub by visiting:
remote:   https://github.com/Sulav-ui/student-web-project/pull/new/feature-footer
remote:
To https://github.com/Sulav-ui/student-web-project.git
 * [new branch]      feature-footer -> feature-footer
branch 'feature-footer' set up to track 'origin/feature-footer'.

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch -r
  origin/development
  origin/feature-contact
  origin/feature-footer
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git fetch
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git switch development
Switched to branch 'development'
Your branch is up to date with 'origin/development'.
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git merge feature-contact
Updating b7784fd..b5f43f2
Fast-forward
 contact.html | 13 ++++++++
1 file changed, 13 insertions(+)
create mode 100644 contact.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -5
b5f43f2 (HEAD -> development, origin/feature-contact, feature-contact) Add contact form
2baf187 Add contact page
b7784fd (origin/development) Fix urgent production bug
d6a14c8 (master, development-old) Add about section
f6106e3 Initial project setup

```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -5
b5f43f2 (HEAD -> development, origin/feature-contact, feature-contact) Add contact form
2baf187 Add contact page
b7784fd (origin/development) Fix urgent production bug
d6a14c8 (master, development-old) Add about section
f6106e3 Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline --graph --all -8
* b22ada5 (origin/feature-footer, feature-footer) Add website footer
| * b5f43f2 (HEAD -> development, origin/feature-contact, feature-contact) Add contact form
| * 2baf187 Add contact page
|/
* b7784fd (origin/development) Fix urgent production bug
| * 33bff75 (refs/stash) On feature-profile: Save profile work before final challenge
| |
| | * 969a3d6 index on feature-profile: 93e95d0 Fix urgent production bug
| | /
| * 93e95d0 (feature-profile) Fix urgent production bug
|/
| * f260f82 (abort-two) Revert "Add temporary shared change"
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git merge feature-footer
Merge made by the 'ort' strategy.
index.html | 3 +--
1 file changed, 1 insertion(+), 2 deletions(-)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline --graph -8
* c2f65da (HEAD -> development) Merge branch 'feature-footer' into development
| \
| * b22ada5 (origin/feature-footer, feature-footer) Add website footer
* | b5f43f2 (origin/feature-contact, feature-contact) Add contact form
* | 2baf187 Add contact page
|/
* b7784fd (origin/development) Fix urgent production bug
* d6a14c8 (master, development-old) Add about section
* f6106e3 Initial project setup
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch development
Your branch is ahead of 'origin/development' by 4 commits.
(use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   contact.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash push -m "Temporary unfinished contact update"
Saved working directory and index state On development: Temporary unfinished contact update
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git stash list
stash@{0}: On development: Temporary unfinished contact update
stash@{1}: On feature-profile: Save profile work before final challenge
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch development
Your branch is ahead of 'origin/development' by 4 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

```

```

PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git add contact.html
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git commit -m "Temporary recovery test"
[development 6fccbd7] Temporary recovery test
 1 file changed, 1 insertion(+)
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -3
6fccbd7 (HEAD -> development) Temporary recovery test
c2f65da Merge branch 'feature-footer' into development
b22ada5 (origin/feature-footer, feature-footer) Add website footer
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git reset --hard HEAD~1
HEAD is now at c2f65da Merge branch 'feature-footer' into development
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log --oneline -3
c2f65da (HEAD -> development) Merge branch 'feature-footer' into development
b22ada5 (origin/feature-footer, feature-footer) Add website footer
b5f43f2 (origin/feature-contact, feature-contact) Add contact form
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git reflog
c2f65da (HEAD -> development) HEAD@{0}: reset: moving to HEAD~1
6fccbd7 HEAD@{1}: commit: Temporary recovery test
c2f65da (HEAD -> development) HEAD@{2}: reset: moving to HEAD
c2f65da (HEAD -> development) HEAD@{3}: merge feature-footer: Merge made by the 'ort' strategy.
b5f43f2 (origin/feature-contact, feature-contact) HEAD@{4}: merge feature-contact: Fast-forward
b7784fd (origin/development) HEAD@{5}: checkout: moving from feature-footer to development
b22ada5 (origin/feature-footer, feature-footer) HEAD@{6}: commit: Add website footer
b7784fd (origin/development) HEAD@{7}: checkout: moving from development to feature-footer
b7784fd (origin/development) HEAD@{8}: checkout: moving from feature-contact to development
b5f43f2 (origin/feature-contact, feature-contact) HEAD@{9}: commit: Add contact form
2baf187 HEAD@{10}: commit: Add contact page
b7784fd (origin/development) HEAD@{11}: checkout: moving from development to feature-contact
b7784fd (origin/development) HEAD@{12}: checkout: moving from feature-profile to development
93e95d0 (feature-profile) HEAD@{13}: reset: moving to HEAD
93e95d0 (feature-profile) HEAD@{14}: checkout: moving from development to feature-profile
b7784fd (origin/development) HEAD@{15}: cherry-pick: Fix urgent production bug
d6a14c8 (master, development-old) HEAD@{16}: checkout: moving from feature-profile to development

```

```
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git branch recovered-final 6fccbd7
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git log recovered-final --oneline -3
6fccbd7 (recovered-final) Temporary recovery test
c2f65da (HEAD -> development) Merge branch 'feature-footer' into development
b22ada5 (origin/feature-footer, feature-footer) Add website footer
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch development
Your branch is ahead of 'origin/development' by 4 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git push origin development
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 316 bytes | 316.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Sulav-ui/student-web-project.git
   b7784fd..c2f65da  development -> development
PS C:\Users\ksula\OneDrive\Desktop\student-web-project> git status
On branch development
Your branch is up to date with 'origin/development'.

nothing to commit, working tree clean
```